Unit V

Bottom up Parsing

Topics:

- handle pruning
- shift reduce parser
- LR Parsers(SLR,CLR,LALR)
- operator precedence parser
- YACC tool.

Bottom-Up Parsing

- A **bottom-up parser** creates the parse tree of the given input starting from leaves towards the root.
- A bottom-up parser tries to find the right-most derivation of the given input in the reverse order.
- One advantage with bottom up parsing is , even left recursive and non left factored grammars are also handled.

Bottom-Up Parsing

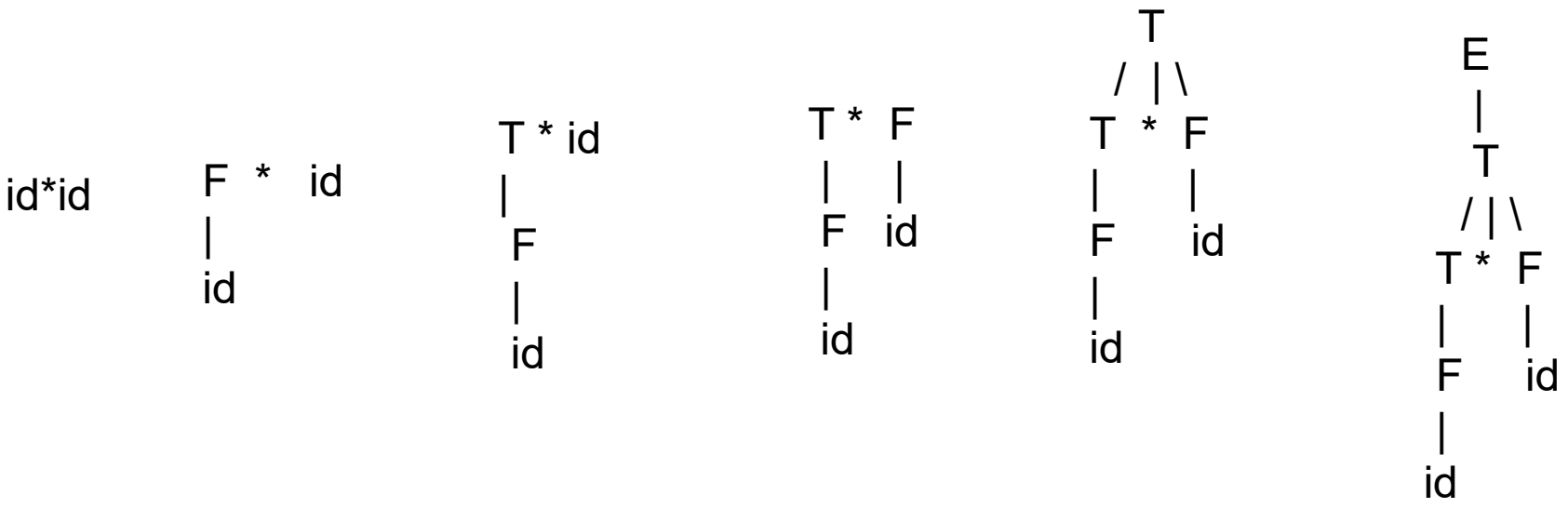
- Consider a grammar:

E->E+T | T

T->T*F | F

F->id | (E)

- Bottom – up parse tree for id*id



Bottom up Parsing -- Example

$S \rightarrow aABb$

$A \rightarrow aA \mid a$

$B \rightarrow bB \mid b$

input string: aaabb

aaAbb

aAbb

aABb

S

Bottom up Parsing -- Example

S → aABb

A → aA | a

B → bB | b

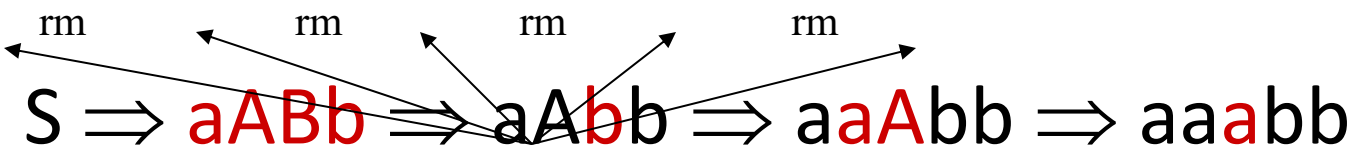
input string: aaabb

aaAbb

aAbb ⇓ reduction

aABb

S



Right Sentential Forms

Bottom-Up Parsing

- A bottom –up parsing is the process of " reducing" an input string to the starting symbol of the grammar.

- At each reduction step , a specific substring matching the body of a production (called handle) is replaced by the corresponding left hand side non terminal.

- If the grammar is unambiguous, then every right-sentential form of the grammar has exactly one handle.

Handle pruning

A left to right scan of the input constructs right-most derivation in reverse can be obtained by **handle-pruning**

Consider a grammar :

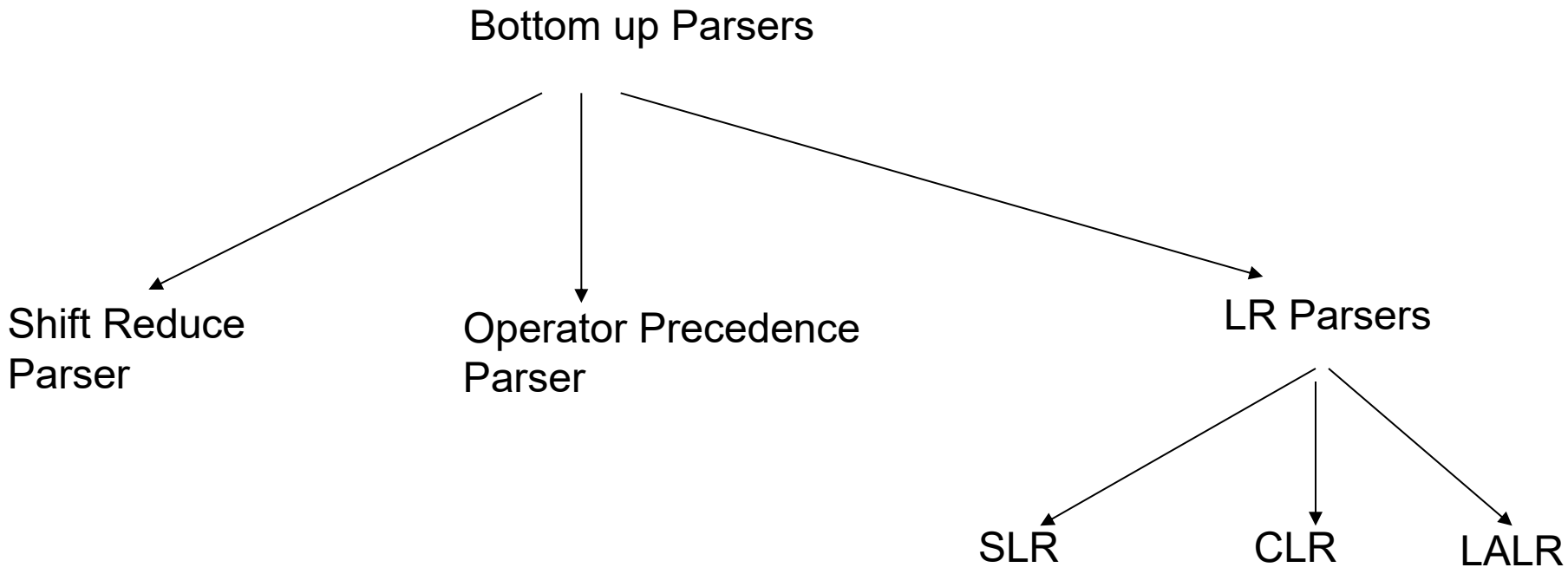
E->E+T | T

T-> T*F |F

F->id | (E)

Input string : id* id

Right-Most Sentential Form	Handle	Reducing Production
id*id	id	F->id
F * id	F	T->F
T*id	id	F->id
T*F	T*F	T->T*F
T	T	E->T
E		


Bottom up Parsers
Shift Reduce Parser
Operator Precedence Parser
LR Parsers
SLR
CLR
LALR

A Stack Implementation of A Shift-Reduce Parser

- There are four possible actions of a shift-parser action:

1. **Shift** : The next input symbol is shifted onto the top of the stack.
2. **Reduce**: Replace the handle on the top of the stack by the non-terminal.
3. **Accept**: Successful completion of parsing.
4. **Error**: Parser discovers a syntax error, and calls an error recovery routine.

- Initial stack just contains only the end-marker $.
- The end of the input string is marked by the end-marker $.

A Stack Implementation of A Shift-Reduce Parser

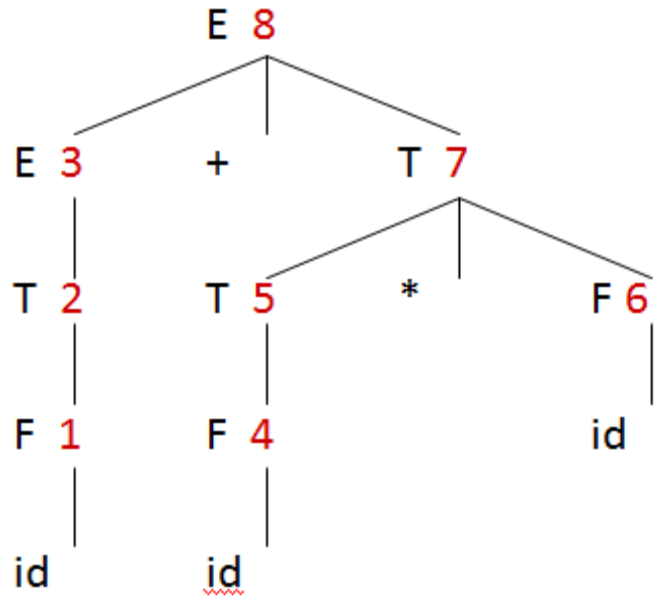
Stack	**Input**	**Action**
$	id+id*id$	shift
$id	+id*id$	reduce by F → id
$F	+id*id$	reduce by T → F
$T	+id*id$	reduce by E → T
$E	+id*id$	shift
$E+	id*id$	shift
$E+id	*id$	reduce by F → id
$E+F	*id$	reduce by T → F
$E+T	*id$	shift
$E+T*	id$	shift
$E+T*id	$	reduce by F → id
$E+T*F	$	reduce by T → T*F
$E+T	$	reduce by E → E+T
$E	$	accept

E->E+T | T
T-> T*F |F
F->id | (E)

A Stack Implementation of A Shift-Reduce Parser

Stack	Input	Action
$	id+id*id$	shift
$id	+id*id$	reduce by F → id
$F	+id*id$	reduce by T → F
$T	+id*id$	reduce by E → T
$E	+id*id$	shift
$E+	id*id$	shift
$E+id	*id$	reduce by F → id
$E+F	*id$	reduce by T → F
$E+T	*id$	shift
$E+T*	id$	shift
$E+T*id	$	reduce by F → id
$E+T*F	$	reduce by T → T*F
$E+T	$	reduce by E → E+T
$E	$	accept

Parse Tree



Conflicts During Shift-Reduce Parsing

- There are context-free grammars for which shift-reduce parsers cannot be used.
- Stack contents and the next input symbol may not decide action:

1. shift/reduce conflict:

Whether make a shift operation or a reduction.

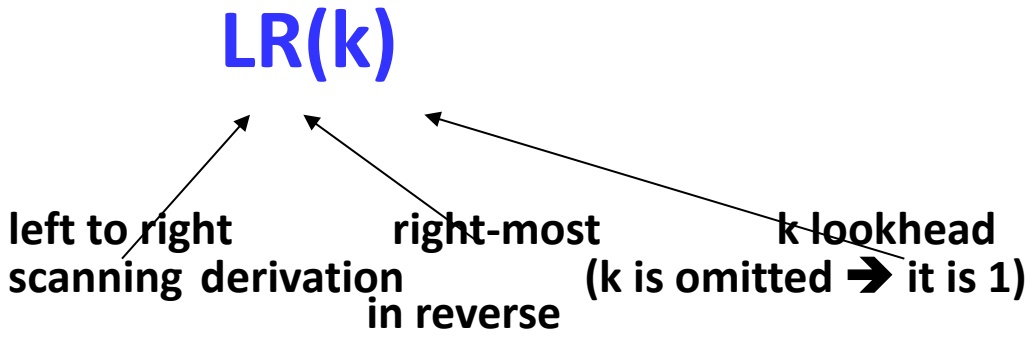
2. reduce/reduce conflict:

The parser cannot decide which of several reductions to make.

That is can't be determine what production to choose in case there is more than one production with that substring on the right side.

LR Parsers

- The most powerful shift-reduce parsing (yet efficient) is:



- LR parsing is attractive because:
  - The class of grammars that can be parsed using LR methods is a proper superset of the class of grammars that can be parsed with predictive parsers.

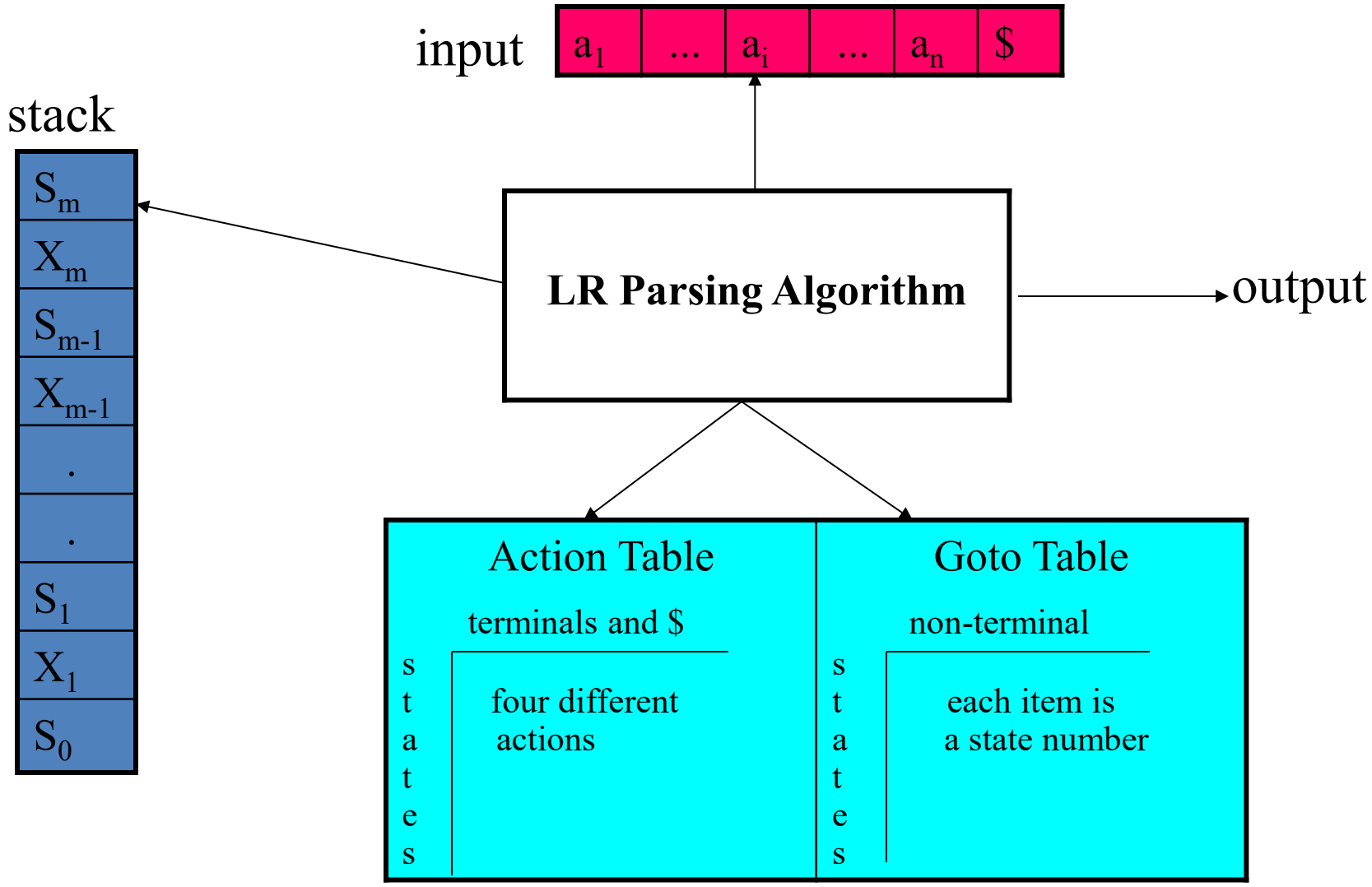
    LL(1)-Grammars ⊂ LR(1)-Grammars
  - An LR-parser can detect a syntactic error as soon as it is possible to do so a left-to-right scan of the input.

- **Types of LR-Parsers**

  - **SLR – Simple LR parser**
  - **CLR – Canonical LR Parser (most powerful LR parser)**
  - **LALR – Look-Ahead LR parser (intermediate LR parser & efficient)**

  SLR, LR and LALR work same (they used the same algorithm), only their parsing tables are different.

LR Parsing Algorithm



A Configuration of LR Parsing Algorithm

- A configuration of a LR parsing is:

$$(S_o\ X_1\ S_1 \ldots X_m\ S_m,\ \ a_i\ a_{i+1} \ldots a_n\ \$)$$

Stack

Rest of Input

- S_m and a_i decides the parser action by consulting the parsing action table. (*Initial Stack* contains just S_o)

- A configuration of a LR parsing represents the right sentential form:

$$X_1 \ldots X_m\ a_i\ a_{i+1} \ldots a_n\ \$$$

Actions of a LR-Parser

1. **shift s** -- shifts the next input symbol and the state **s** onto the stack

   ($S_o X_1 S_1 \dots X_m S_m$, $a_i a_{i+1} \dots a_n \$$) ➔ ($S_o X_1 S_1 \dots X_m S_m a_i s$, $a_{i+1} \dots a_n \$$)

2. **reduce $A \rightarrow \beta$** (or **`rn`** where n is a production number)
   - pop $2|\beta|$ (=r) items from the stack;
   - then push **A** and **s** where **$s=goto[s_{m-r},A]$**

   ($S_o X_1 S_1 \dots X_m S_m$, $a_i a_{i+1} \dots a_n \$$) ➔ ($S_o X_1 S_1 \dots X_{m-r} S_{m-r} A s$, $a_i \dots a_n \$$)

   - Output is the reducing production reduce $A \rightarrow \beta$

3. **Accept** – Parsing successfully completed

4. **Error** -- Parser detected an error (an empty entry in the action table)

Reduce Action

- pop $2|\beta|$ (=r) items from the stack; let us assume that $\beta = Y_1 Y_2 \ldots Y_r$
- then push **A** and **s** where **$s=goto[s_{m-r}, A]$**

$(S_o X_1 S_1 \ldots X_{m-r} S_{m-r} Y_1 S_{m-r} \ldots Y_r S_m, a_i a_{i+1} \ldots a_n \$)$

$\rightarrow (S_o X_1 S_1 \ldots X_{m-r} S_{m-r} A s, a_i \ldots a_n \$)$

- In fact, $Y_1 Y_2 \ldots Y_r$ is a handle.

$X_1 \ldots X_{m-r} A a_i \ldots a_n \$ \Rightarrow X_1 \ldots X_m Y_1 \ldots Y_r a_i a_{i+1} \ldots a_n \$$

(SLR) Parsing Tables for Expression Grammar

1) $E \rightarrow E+T$
2) $E \rightarrow T$
3) $T \rightarrow T*F$
4) $T \rightarrow F$
5) $F \rightarrow (E)$
6) $F \rightarrow id$

	Action Table							Goto Table		
state	**id**	**+**	*****	**(**	**)**	**$**		**E**	**T**	**F**
0	**s5**			**s4**				**1**	**2**	**3**
1		**s6**				**acc**				
2		**r2**	**s7**		**r2**	**r2**				
3		**r4**	**r4**		**r4**	**r4**				
4	**s5**			**s4**				**8**	**2**	**3**
5		**r6**	**r6**		**r6**	**r6**				
6	**s5**			**s4**					**9**	**3**
7	**s5**			**s4**						**10**
8		**s6**			**s11**					
9		**r1**	**s7**		**r1**	**r1**				
10		**r3**	**r3**		**r3**	**r3**				
11		**r5**	**r5**		**r5**	**r5**				

Actions of a (S)LR-Parser -- Example

stack	input	action	output
0	id*id+id$	shift 5	
0id5	*id+id$	reduce by F→id	F→id
0F3	*id+id$	reduce by T→F	T→F
0T2	*id+id$	shift 7	
0T2*7	id+id$	shift 5	
0T2*7id5	+id$	reduce by F→id	F→id
0T2*7F10	+id$	reduce by T→T*F	T→T*F
0T2	+id$	reduce by E→T	E→T
0E1	+id$	shift 6	
0E1+6	id$	shift 5	
0E1+6id5	$	reduce by F→id	F→id
0E1+6F3	$	reduce by T→F	T→F
0E1+6T9	$	reduce by E→E+T	E→E+T
0E1	$	accept	

Constructing SLR Parsing Tables – LR(0) Item

- An **LR(0) item** of a grammar G is a production of G a dot at the some position of the right side.
- Ex: $A \rightarrow aBb$ Possible LR(0) Items: $A \rightarrow .aBb$

  (four different possibility) $A \rightarrow a.Bb$

  $A \rightarrow aB.b$

  $A \rightarrow aBb.$
- Sets of LR(0) items will be the states of action and goto table of the SLR parser.
- A collection of sets of LR(0) items (**the canonical LR(0) collection**) is the basis for constructing SLR parsers.
- *Augmented Grammar*:

  G' is G with a new production rule $S' \rightarrow S$ where S' is the new starting symbol.

The Closure Operation

- If ***I*** is a set of LR(0) items for a grammar G, then ***closure(I)*** is the set of LR(0) items constructed from I by the two rules:

  1. Initially, every LR(0) item in I is added to closure(I).
  2. If $A \rightarrow \alpha \bullet B\beta$ is in closure(I) and $B \rightarrow \gamma$ is a production rule of G; then $B \rightarrow \bullet \gamma$ will be in the closure(I).
     We will apply this rule until no more new LR(0) items can be added to closure(I).

The Closure Operation -- Example

E’ → E
E → E+T
E → T
T → T*F
T → F
F → (E)
F → id

closure({E’ → .E}) =
{ E’ → .E
E → .E+T
E → .T
T → .T*F
T → .F
F → .(E)
F → .id }

Goto Operation

- If I is a set of LR(0) items and X is a grammar symbol (terminal or non-terminal), then goto(I,X) is defined as follows:
  - If $A \rightarrow \alpha \bullet X\beta$ in I then every item in **closure({$A \rightarrow \alpha X \bullet \beta$})** will be in goto(I,X).

Example:

I ={ $E' \rightarrow \bullet E$, $E \rightarrow \bullet E+T$, $E \rightarrow \bullet T$,
$T \rightarrow \bullet T*F$, $T \rightarrow \bullet F$,
$F \rightarrow \bullet (E)$, $F \rightarrow \bullet id$ }

goto(I,E) = { $E' \rightarrow E \bullet$, $E \rightarrow E \bullet +T$ }

goto(I,T) = { $E \rightarrow T \bullet$, $T \rightarrow T \bullet *F$ }

goto(I,F) = {$T \rightarrow F \bullet$ }

goto(I,() = { $F \rightarrow (\bullet E)$, $E \rightarrow \bullet E+T$, $E \rightarrow \bullet T$, $T \rightarrow \bullet T*F$, $T \rightarrow \bullet F$,
$F \rightarrow \bullet (E)$, $F \rightarrow \bullet id$ }

goto(I,id) = { $F \rightarrow id \bullet$ }

Construction of The Canonical LR(0) Collection

- To create the SLR parsing tables for a grammar G, we will create the canonical LR(0) collection of the grammar G'.

- ***Algorithm***:

  C is { closure({S'$\rightarrow \bullet$ S}) }

  repeat the followings until no more set of LR(0) items can be added to ***C***.

  for each I in ***C*** and each grammar symbol X

  if goto(I,X) is not empty and not in ***C***

  add goto(I,X) to ***C***

- goto function is a DFA on the sets in C.

The Canonical LR(0) Collection

Closure($E' \to .E$)

I_0: $E' \to .E$
$E \to .E+T$
$E \to .T$
$T \to .T*F$
$T \to .F$
$F \to .(E)$
$F \to .id$

Goto(I_0,E)

I_1: $E' \to E.$
$E \to E.+T$

Goto(I_0,T)

I_2: $E \to T.$
$T \to T.*F$

Goto(I_0,F)

I_3: $T \to F.$

Goto(I_0,( )

I_4: $F \to (.E)$
$E \to .E+T$
$E \to .T$
$T \to .T*F$
$T \to .F$
$F \to .(E)$
$F \to .id$

Goto(I_0,id)

I_5: $F \to id.$

Goto(I_1,+)

I_6: $E \to E+.T$
$T \to .T*F$
$T \to .F$
$F \to .(E)$
$F \to .id$

Goto(I_2,*)

I_7: $T \to T*.F$
$F \to .(E)$
$F \to .id$

Goto(I_4, E)

I_8: $F \to (E.)$
$E \to E.+T$

Goto(I_6,T)

I_9: $E \to E+T.$
$T \to T.*F$

Goto(I_7,F)

I_{10}: $T \to T*F.$

Goto(I_8,))

I_{11}: $F \to (E).$

Constructing SLR Parsing Table

(of an augumented grammar G')

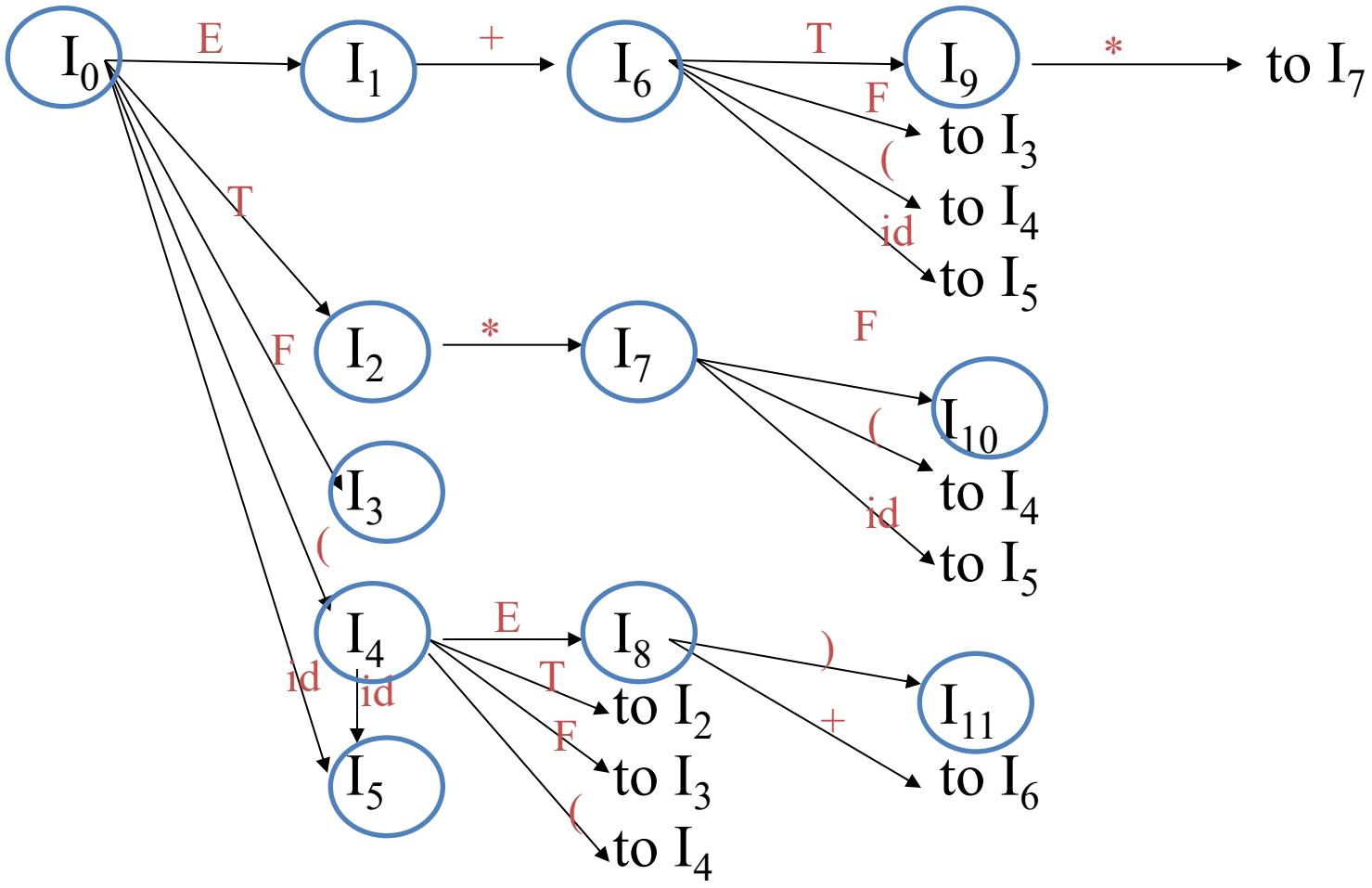
1. Construct the canonical collection of sets of LR(0) items for G'. $C \leftarrow \{I_0,\ldots,I_n\}$
2. Create the parsing action table as follows
   - If a is a terminal, $A \rightarrow \alpha.a\beta$ in I_i and goto(I_i,a)=I_j then action[i,a] is ***shift j.***
   - If $A \rightarrow \alpha.$ is in I_i, then action[i,a] is ***reduce*** $A \rightarrow \alpha$ for all a in FOLLOW(A) where $A \neq S'$.
   - If $S' \rightarrow S.$ is in I_i, then action[i,$] is ***accept***.
   - If any conflicting actions generated by these rules, the grammar is not SLR(1).
3. Create the parsing goto table
   - for all non-terminals A, if goto(I_i,A)=I_j then goto[i,A]=j
4. All entries not defined by (2) and (3) are errors.
5. Initial state of the parser contains $S' \rightarrow .S$

Parsing Tables of Expression Grammar

Action Table | Goto Table

state	id	+	*	(	)	$		E	T	F
0	s5			s4				1	2	3
1		s6				acc				
2		r2	s7		r2	r2				
3		r4	r4		r4	r4				
4	s5			s4				8	2	3
5		r6	r6		r6	r6				
6	s5			s4					9	3
7	s5			s4						10
8		s6			s11					
9		r1	s7		r1	r1				
10		r3	r3		r3	r3				
11		r5	r5		r5	r5				

Transition Diagram (DFA) of Goto Function



SLR(1) Grammar

- An LR parser using SLR(1) parsing tables for a grammar G is called as the SLR(1) parser for G.
- If a grammar G has an SLR(1) parsing table, it is called SLR(1) grammar (or SLR grammar in short).
- Every SLR grammar is unambiguous, but every unambiguous grammar is not a SLR grammar.

shift/reduce and *reduce/reduce* conflicts

- If a state does not know whether it will make a shift operation or reduction for a terminal, we say that there is a **shift/reduce conflict**.

- If a state does not know whether it will make a reduction operation using the production rule `i` or `j` for a terminal, we say that there is a **reduce/reduce conflict**.

- If the SLR parsing table of a grammar G has a conflict, we say that that grammar is not SLR grammar.

Conflict Example

S → L=R
S → R
L→ *R
L → id
R → L

I_0: S' → .S
S → .L=R
S → .R
L → .*R
L → .id
R → .L

I_1: S' → S.

I_2: S → L.=R
R → L.

I_3: S → R.

I_4: L → *.R
R → .L
L→ .*R
L → .id

I_5: L → id.

I_6: S → L=.R
R → .L
L→ .*R
L → .id

I_7: L → *R.

I_8: R → L.

I_9: S → L=R.

Problem

FOLLOW(R)={=,$}

= → shift 6

→ reduce by R → L

shift/reduce conflict

SLR parsing

state	action				goto		
	id	=	*	$	S	L	R
0	s3		s5		1	2	4
1				accept			
2		s6/r(R→L)					
3		r(L→id)		r(L→id)			
4				r(S→R)			
5	s3		s5			7	8
6	s3		s5			7	9
7		r(R→L)		r(R→L)			
8		r(L→*R)		r(L→*R)			
9				r(S→L=R)			

Note the shift/reduce conflict on state 2 when the lookahead is an =

Conflict Example2

$S \rightarrow AaAb$

$S \rightarrow BbBa$

$A \rightarrow \varepsilon$

$B \rightarrow \varepsilon$

I_0: $S' \rightarrow .S$

$S \rightarrow .AaAb$

$S \rightarrow .BbBa$

$A \rightarrow .$

$B \rightarrow .$

Problem

FOLLOW(A)={a,b}

FOLLOW(B)={a,b}

a → reduce by $A \rightarrow \varepsilon$

a → reduce by $B \rightarrow \varepsilon$

reduce/reduce conflict

b → reduce by $A \rightarrow \varepsilon$

b → reduce by $B \rightarrow \varepsilon$

reduce/reduce conflict

LR(1) Item (CLR Parsing)

- To avoid some of invalid reductions, the states need to carry more information.
- Extra information is put into a state by including a terminal symbol as a second component in an item.
- A LR(1) item is:

  $\mathbf{A \rightarrow \alpha . \beta \ , \ a}$ where **a** is the look-head of the LR(1) item

  (**a** is a terminal or end-marker.)

LR(1) Item (cont.)

- When β (in the LR(1) item $A \rightarrow \alpha.\beta,a$) is not empty, the look-head does not have any affect.
- When β is empty ($A \rightarrow \alpha.,a$), we do the reduction by $A \rightarrow \alpha$ only if the next input symbol is **a** (not for any terminal in FOLLOW(A)).
- A state will contain $A \rightarrow \alpha.,a_1$ where $\{a_1,...,a_n\} \subseteq$ FOLLOW(A)

...

$A \rightarrow \alpha.,a_n$

Canonical Collection of Sets of LR(1) Items

- The construction of the canonical collection of the sets of LR(1) items are similar to the construction of the canonical collection of the sets of LR(0) items, except that *closure* and *goto* operations work a little bit different.

closure(I) is: (where I is a set of LR(1) items)

- every LR(1) item in I is in closure(I)
- if $A \rightarrow \alpha . B\beta, a$ in closure(I) and $B \rightarrow \gamma$ is a production rule of G; then $B \rightarrow .\gamma, b$ will be in the closure(I) for each terminal b in **FIRST(βa)**

.

goto operation

- If I is a set of LR(1) items and X is a grammar symbol (terminal or non-terminal), then goto(I,X) is defined as follows:
  - If $A \rightarrow \alpha.X\beta,a$ in I
    then every item in **closure({$A \rightarrow \alpha X.\beta,a$})** will be in goto(I,X).

Construction of The Canonical LR(1) Collection

- ***Algorithm***:

  C is { closure({S'$\rightarrow$.S,$}) }

  repeat the followings until no more set of LR(1) items can be added to ***C***.

  for each I in ***C*** and each grammar symbol X

  if goto(I,X) is not empty and not in ***C***

  add goto(I,X) to ***C***

- goto function is a DFA on the sets in C.

A Short Notation for The Sets of LR(1) Items

- A set of LR(1) items containing the following items

$$A \rightarrow \alpha.\beta,a_1$$

$$\ldots$$

$$A \rightarrow \alpha.\beta,a_n$$

can be written as

$$A \rightarrow \alpha.\beta,a_1/a_2/\ldots/a_n$$

EXAMPLE

GRAMMAR:

S’ -> S

S -> CC

C -> cC

C -> d

SET OF LR (1) ITEMS:

I_0 : goto(S^1 -> .S , $)

S^1 -> .S , $
S -> .CC , $
C -> .c C , c /d
C -> .d , c /d

I_1: goto(I_0 ,S)
S^1 -> S. , $

I_2: goto(I_0 , C)
S -> C.C, $
C -> .cC, $
C -> .d, $

I_3: goto(I_0 , c)
C -> c. C , c /d
C -> .cC , c /d
C -> .d , c /d

I_4: goto(I_0 , d)
C -> d. , c /d

I_5: goto(I_2 , C)
S -> CC., $

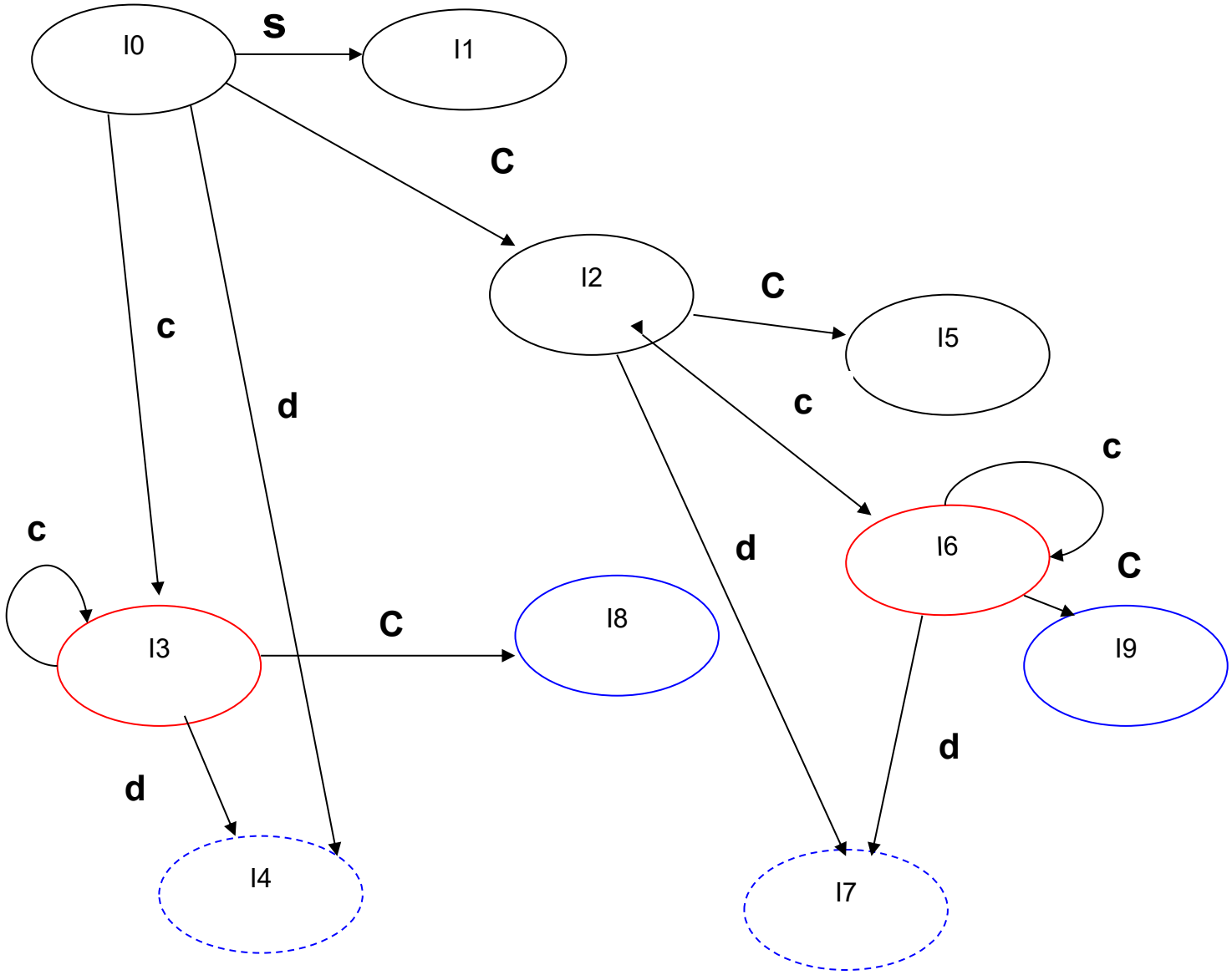
I_6: goto(I_2 , c)
C -> c.C, $
C -> .cC, $
C -> .d, $

I_7: goto(I_2 , d)
C -> d., $

I_8: goto(I_3 ,C)
C -> cC., c /d

I_9: goto(I_6 ,C)
C -> cC., $

GOTO graph (DFA)



Construction of LR(1) Parsing Tables

1. Construct the canonical collection of sets of LR(1) items for G'. $C \leftarrow \{I_0,...,I_n\}$
2. Create the parsing action table as follows
   - If a is a terminal, $A \rightarrow \alpha \bullet a\beta,b$ in I_i and $goto(I_i,a)=I_j$ then action[i,a] is ***shift j.***
   - If $A \rightarrow \alpha \bullet,a$ is in I_i, then action[i,a] is ***reduce*** $A \rightarrow \alpha$ where $A \neq S'$.
   - If $S' \rightarrow S \bullet,\$$ is in I_i, then action[i,$] is ***accept***.
   - If any conflicting actions generated by these rules, the grammar is not LR(1).
3. Create the parsing goto table
   - for all non-terminals A, if $goto(I_i,A)=I_j$ then goto[i,A]=j
4. All entries not defined by (2) and (3) are errors.
5. Initial state of the parser contains $S' \rightarrow .S,\$$

CLR PARSING TABLE

STATE	Actions			Goto	
	c	**d**	**$**	**S**	**C**
0	**S3**	**S4**		**1**	**2**
1			**acc**		
2	**S6**	**S7**			**5**
3	**S3**	**S4**			**8**
4	**R3**	**R3**			
5			**R1**		
6	**S6**	**S7**			**9**
7			**R3**		
8	**R2**	**R2**			
9			**R2**		

LALR Parsing Tables

- **LALR** stands for **LookAhead LR.**
- LALR parsers are often used in practice because LALR parsing tables are smaller than LR(1) parsing tables.
- The number of states in SLR and LALR parsing tables for a grammar G are equal.
- But LALR parsers recognize more grammars than SLR parsers.
- **YACC** (Yet Another Compiler Compiler) creates a LALR parser for the given grammar.
- A state of LALR parser will be again a set of LR(1) items.

Creating LALR Parsing Tables

Canonical LR(1) Parser ➔LALR Parser

shrink # of states

- This shrink process may introduce a **reduce/reduce** conflict in the resulting LALR parser (so the grammar is NOT LALR)
- But, this shrink process does not produce a **shift/reduce** conflict.

The Core of a Set of LR(1) Items

- The core of a set of LR(1) items is the set of its first component.

Ex:

C -> c. C , c /d
C -> .cC , c /d
C -> .d , c /d

➔

C -> c. C
C -> .cC
C -> .d

Core

We will find the states (sets of LR(1) items) in a canonical LR(1) parser with same cores. Then we will merge them as a single state.

I_3: **C -> c. C , c /d**
C -> .cC , c /d
C -> .d , c /d

I_6 : **C -> c.C, $**
C -> .cC, $
C -> .d, $

- I3 and I6 have same core , merge them

I_{36}: **C -> c. C , c /d/$**
C -> .cC , c /d/$
C -> .d , c /d/$

- We will do this for all states of a canonical LR(1) parser to get the states of the LALR parser.
- In fact, the number of the states of the LALR parser for a grammar will be equal to the number of states of the SLR parser for that grammar.

Creation of LALR Parsing Tables

- Create the canonical LR(1) collection of the sets of LR(1) items for the given grammar.
- Find each core; find all sets having that same core; replace those sets having same cores with a single set which is their union.

  $C=\{I_0,...,I_n\} \rightarrow C'=\{J_1,...,J_m\}$ where $m \leq n$
- Create the parsing tables (action and goto tables) same as the construction of the parsing tables of LR(1) parser.
  - Note that: If $J=I_1 \cup ... \cup I_k$ since $I_1,...,I_k$ have same cores

    $\rightarrow$ cores of $goto(I_1,X),...,goto(I_2,X)$ must be same.
  - So, $goto(J,X)=K$ where K is the union of all sets of items having same cores as $goto(I_1,X)$.
- If no conflict is introduced, the grammar is LALR(1) grammar.(We may only introduce reduce/reduce conflicts; we cannot introduce a shift/reduce conflict)

LALR PARSING TABLE

STATE	Actions			goto	
	C	**D**	**$**	**S**	**C**
0	**S36**	**S47**		**1**	**2**
1			**Acc**		
2	**S36**	**S47**			**5**
36	**S36**	**S47**			**89**
47	**R3**	**R3**	**R3**		
5			**R1**		
89	**R2**	**R2**	**R2**		

Construct SLR parsing table for the grammar below:

1. $E \rightarrow aEbE$

$E \rightarrow bEaE$

$E \rightarrow \epsilon$

2. $S \rightarrow B$

$S \rightarrow SabS$

$B \rightarrow bB$

$B \rightarrow \epsilon$

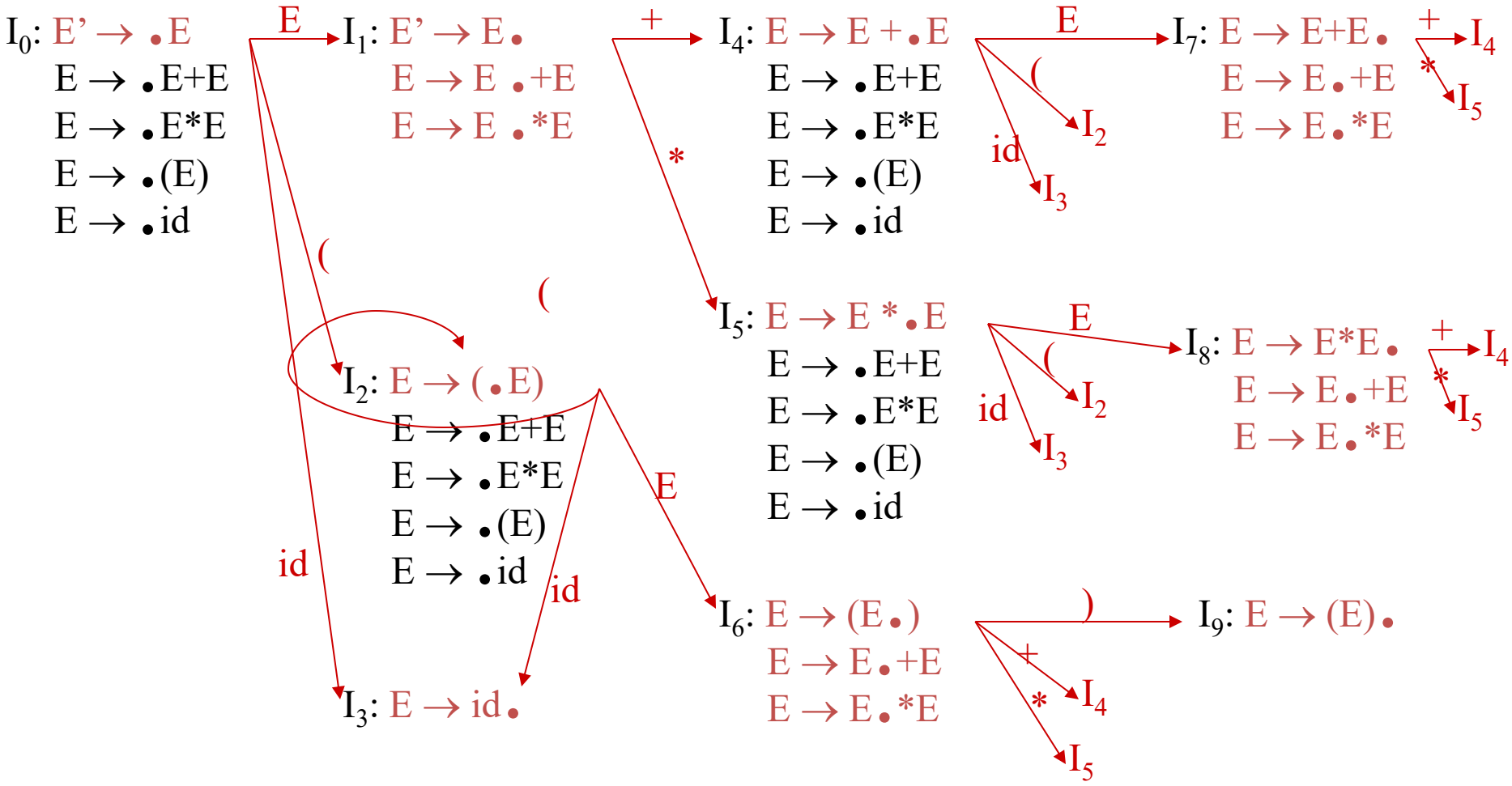
Using Ambiguous Grammars

- All grammars used in the construction of LR-parsing tables must be un-ambiguous.
- Can we create LR-parsing tables for ambiguous grammars ?
  - Yes, but they will have conflicts.
  - We can resolve these conflicts in favor of one of them to disambiguate the grammar.
  - At the end, we will have again an unambiguous grammar.
- Why we want to use an ambiguous grammar?
  - Some of the ambiguous grammars are **much natural**, and a corresponding unambiguous grammar can be very complex.
  - Usage of an ambiguous grammar may **eliminate unnecessary reductions**.
- Ex.

$E \rightarrow E+E \mid E*E \mid (E) \mid id$ ➔

$E \rightarrow E+T \mid T$
$T \rightarrow T*F \mid F$
$F \rightarrow (E) \mid id$

Sets of LR(0) Items for Ambiguous Grammar



SLR-Parsing Tables for Ambiguous Grammar

FOLLOW(E) = { $, +, *,) }

State I_7 has shift/reduce conflicts for symbols + and *.

$$I_0 \xrightarrow{E} I_1 \xrightarrow{+} I_4 \xrightarrow{E} I_7$$

when current token is +

shift ➔ + is right-associative

reduce ➔ + is left-associative

when current token is *

shift ➔ * has higher precedence than +

reduce ➔ + has higher precedence than *

SLR-Parsing Tables for Ambiguous Grammar

FOLLOW(E) = { $, +, *,) }

State I_8 has shift/reduce conflicts for symbols + and *.

$$I_0 \xrightarrow{E} I_1 \xrightarrow{*} I_5 \xrightarrow{E} I_7$$

when current token is *

shift ➔ * is right-associative

reduce ➔ * is left-associative

when current token is +

shift ➔ + has higher precedence than *

reduce ➔ * has higher precedence than +

SLR-Parsing Tables for Ambiguous Grammar

	Action							Goto
	id	+	*	(	)	$		**E**
0	s3			s2				1
1		s4	s5			acc		
2	s3			s2				6
3		r4	r4		r4	r4		
4	s3			s2				7
5	s3			s2				8
6		s4	s5		s9			
7		r1	s5		r1	r1		
8		r2	r2		r2	r2		
9		r3	r3		r3	r3		

Operator Precedence Parsing

- Operator precedence parsing is used for only small class of grammars.
- This parsing technique uses *operator grammars.*
- A grammar with no production right side has two adjacent nonterminals or has $\in$ is said to be operator grammar.

Example: S-> ABa | S

A->b

B->c

- The above grammar is not an operator grammar. Because the right side production ABa has two consecutive nonterminals.
- This parsing method is implemented through precedence relations

Precedence Relations

- In operator-precedence parsing, we define three disjoint precedence relations (**<·** , **·>** , **≐**) between certain pairs of terminals.

a **<·** b	a "yields precedence to" b (b has higher precedence than a)
a ≐ b	a "has the same precedence as" b
a **·>** b	a "takes precedence over" b (b has lower precedence than a)

- The determination of correct precedence relations between terminals are based on the traditional notions of associativity and precedence of operators.
- The intention of the precedence relations is to find the handle of a right-sentential form.

Using Operator -Precedence Relations

$E \rightarrow E+E \mid E*E \mid id$

The operator-precedence table for this grammar

	id	+	*	$
id		·>	·>	·>
+	<·	·>	<·	·>
*	<·	·>	·>	·>
$	<·	<·	<·	

- Then the input string **id+id*id** with the precedence relations inserted will be:

  $ <· id ·> + <· id ·> * <· id ·> $

To Find The Handles

1. Scan the string from left end until the first ·> is encountered.
2. Then scan backwards (to the left) over any =· until a <· is encountered.
3. The handle contains everything to left of the first ·> and to the right of the <· is encountered.

$ <· id ·> + <· id ·> * <· id ·> $
$ <· + <· id ·> * <· id ·> $
$ <· + <· * <· id ·> $
$ <· + <· * ·> $
$ <· + ·> $
$ $

E → id
E → id
E → id
E → E*E
E → E+E

$ id + id * id $
$ E + id * id $
$ E + E * id $
$ E + E * ·E $
$ E + E $
$ E $

How to Create Operator-Precedence Relations

- We use associativity and precedence relations among operators.

1. If operator θ_1 has higher precedence than operator θ_2,
   ➔ $\theta_1 \cdot> \theta_2$ and $\theta_2 <\cdot \theta_1$

2. If operator θ_1 and operator θ_2 have equal precedence,
   they are left-associative ➔ $\theta_1 \cdot> \theta_2$ and $\theta_2 \cdot> \theta_1$
   they are right-associative ➔ $\theta_1 <\cdot \theta_2$ and $\theta_2 <\cdot \theta_1$

3. For all operators θ, $\theta <\cdot$ id, id $\cdot> \theta$, $\theta <\cdot$ (, (<· θ, $\theta \cdot>$),) $\cdot> \theta$, $\theta \cdot>$ \$, and \$ <· θ

4. Also, let

($\doteq$)	\$ <· (	id ·>)	) ·> \$
(<· (	\$ <· id	id ·> \$	) ·>)
(<· id			

Operator-Precedence Relations

$E \rightarrow E+E \mid E-E \mid E*E \mid E/E \mid E\hat{}E \mid (E) \mid -E \mid id$

	+	-	*	/	^	id	(	)	$
+	·>	·>	<·	<·	<·	<·	<·	·>	·>
-	·>	·>	<·	<·	<·	<·	<·	·>	·>
*	·>	·>	·>	·>	<·	<·	<·	·>	·>
/	·>	·>	·>	·>	<·	<·	<·	·>	·>
^	·>	·>	·>	·>	<·	<·	<·	·>	·>
id	·>	·>	·>	·>	·>			·>	·>
(	<·	<·	<·	<·	<·	<·	<·	≐	
)	·>	·>	·>	·>	·>			·>	·>
$	<·	<·	<·	<·	<·	<·	<·		

Handling Unary Minus

- Operator-Precedence parsing cannot handle the unary minus when we also have the binary minus in our grammar.
- The best approach to solve this problem, let the lexical analyzer handle this problem.
  - The lexical analyzer will return two different tokens for the unary minus and the binary minus.
  - The lexical analyzer will need a lookhead to distinguish the binary minus from the unary minus.
- Then, we make

$\theta \lessdot$ unary-minus for any operator

unary-minus $\gtrdot \theta$ if unary-minus has higher precedence than θ

unary-minus $\lessdot \theta$ if unary-minus has lower (or equal) precedence than θ

Precedence Functions

- Compilers using operator precedence parsers do not need to store the table of precedence relations.
- The table can be encoded by two precedence functions **f** and **g** that map terminal symbols to integers.
- For symbols a and b.

  f(a) < g(b) whenever a $<\cdot$ b

  f(a) = g(b) whenever a $\doteq$ b

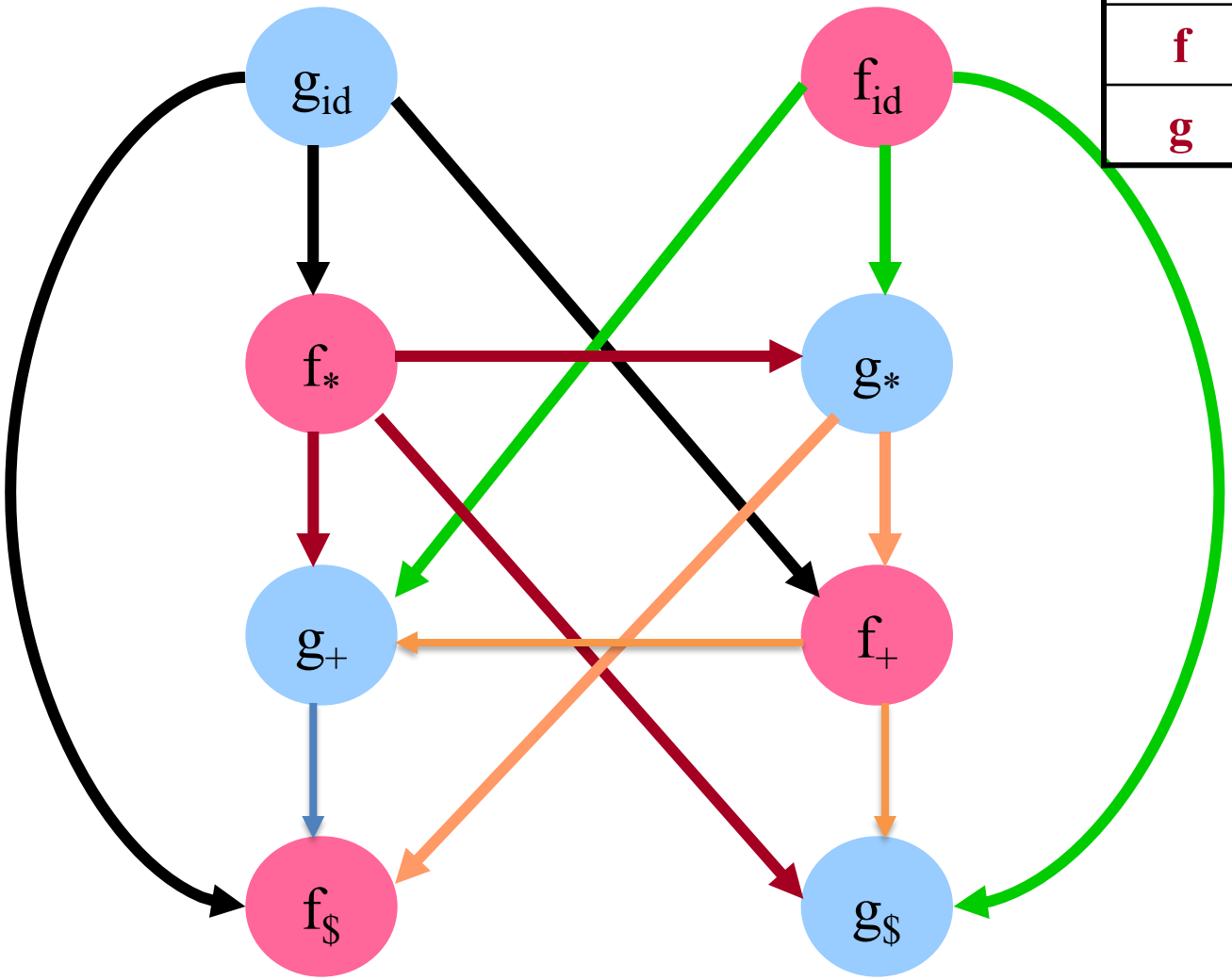
  f(a) > g(b) whenever a $\cdot>$ b

Constructing precedence functions

Method:

1. Create symbols f_a and g_a for each **a** that is a terminal or $.
2. Partition the created symbols into as many groups as possible, in such a way that if a =. b, then f_a and g_b are in the same group.
3. Create a directed graph whose nodes are the groups found in (2). For any a and b, if **a <.b** , place an edge from the group of g_b to the group of f_a. If **a .> b**, place an edge from the group of f_a to that of g_b.
4. If the graph constructed in (3) has a ***cycle***, then no precedence functions exist. If there are no cycle, **let f(a) be the length of the longest path beginning at the group of f_a**; let g(a) be the length of the longest path beginning at the group of g_a.

Example



Precedence function table

	+	*	Id	$
f	2	4	4	0
g	1	3	5	0

Precedence relation table

	id	+	*	$
id		·>	·>	·>
+	<·	·>	<·	·>
*	<·	·>	·>	·>
$	<·	<·	<·	

Disadvantages of Operator Precedence Parsing

- **Disadvantages**:
  - It cannot handle the unary minus (the lexical analyzer should handle the unary minus).
  - Small class of grammars.
  - Difficult to decide which language is recognized by the grammar.

- **Advantages**:
  - simple
  - powerful enough for expressions in programming languages

Parser Generator

. Some common parser generators

- YACC: **Y** et **A** nother **C** ompiler **C** ompiler
- Bison: GNU Software
- ANTLR: **AN** other **T** ool for **L** anguage **R** ecognition

. Yacc/Bison source program specification (accept LALR grammars)

declaration

%%

translation rules

%%

supporting C routines

Yacc and Lex schema